

Progettazione di Algoritmi
Guida ragionata per puntare al 30 e lode

Piano intensivo, schemi, Python, esercizi e soluzioni

Basata sul diario delle lezioni del corso 2025-2026

Questa guida seleziona il nucleo realmente importante del corso e lo organizza in un percorso pratico. Per ogni algoritmo bisogna saper spiegare idea, correttezza, complessità e implementazione. Le slide restano materiale di consultazione: il testo qui presente e la traccia operativa per lo studio.

Come usare questa guida

Con il livello iniziale 6/10 e nessuna esperienza di implementazione Python, il tempo disponibile è circa 41 ore: una ora nei giorni feriali e quattro ore il sabato e la domenica, fino all'esame dell'11 settembre.

Ogni sessione deve produrre un risultato: codice, pseudocodice, dimostrazione o esercizio corretto. Non leggere prima le soluzioni. Per ogni esercizio usa questa scheda: idea; dati e invarianti; pseudocodice; correttezza; complessità; codice; test limite.

Priorità assolute: DFS/BFS, DAG e SCC, Dijkstra, Kruskal, divide et impera, programmazione dinamica e backtracking. Secondarie ma da ripassare: dettagli degli algoritmi di approssimazione e varianti meno frequenti.

Calendario fino all'esame

Studiare rappresentazioni, $O(n+m)$, DFS, componenti, cicli, ponti e alberi. Scrivere almeno connessione, componenti e verifica di albero.

Ordinamento topologico, ciclo diretto, BFS, distanze, cammini e diametro. Fare una sessione lunga per implementare e testare tutti gli algoritmi.

Dijkstra, Bellman-Ford, Floyd-Warshall, Kruskal, Prim, Union-Find e prove di correttezza. Applicare gli algoritmi a grafi disegnati a mano.

Ricerca binaria, massimo sottosegmento, selezione, punti più vicini, ricorrenze e rapporto di approssimazione.

Stato, ricorrenza, basi, ordine, ricostruzione. Zaino, LCS, palindromi, matrici, piastrellamenti e Catalano.

Stringhe, permutazioni, n-regine, Hamiltoniano e zaino. Domenica svolgere una prova completa senza consultare appunti.

Rifare gli errori, ripetere le dimostrazioni, due esercizi brevi al giorno. Il 10 settembre niente argomenti nuovi.

Fondamentali di analisi

Un algoritmo deve essere valutato con correttezza e costo. Scrivere sempre la complessità nella rappresentazione scelta. Con liste di adiacenza si usa n per i nodi e m per gli archi; scorrere tutto il grafo costa $O(n+m)$.

Regole rapide: sequenze sommano i costi; cicli annidati spesso moltiplicano; un ciclo che dimezza il problema costa $O(\log n)$; ricorsione divide in due meta e combina in tempo lineare: $T(n)=2T(n/2)+O(n)=O(n \log n)$.

Per dimostrare correttezza usare induzione, invariante di ciclo o una proprietà strutturale. Una buona risposta d'esame non dice solo "funziona": spiega perché ogni soluzione ammessa viene considerata e perché nessuna migliore viene scartata.

Grafi: il nucleo da sapere

Lista di adiacenza: grafo[u] contiene i vicini di u. Matrice di adiacenza: test di un arco $O(1)$, ma spazio $O(n^2)$. Liste: spazio $O(n+m)$, preferite nei problemi del corso.

Marca un nodo, visita ricorsivamente i vicini non visitati. Serve per componenti, cicli, ponti, punti di articolazione, ordinamenti topologici e SCC. Costo $O(n+m)$.

Usa una coda e visita per livelli. La distanza assegnata al primo ingresso e la distanza minima in un grafo non pesato. Costo $O(n+m)$. Per ricostruire un cammino salva padre[v] quando scopri v.

Un ordinamento e una sequenza in cui ogni arco va da sinistra a destra. Esiste se e solo se il grafo e aciclico. Kahn inserisce in coda i nodi con grado entrante zero; se non estrae n nodi esiste un ciclo. Costo $O(n+m)$.

Le componenti fortemente connesse sono massimali insiemi raggiungibili in entrambe le direzioni. Kosaraju: DFS su G, DFS su G trasposto in ordine inverso dei tempi di fine. Costo $O(n+m)$.

Con tempi di scoperta tin e valore low: un arco dell'albero DFS (u,v) e ponte se $low[v] > tin[u]$. Un nodo u e articolazione se esiste figlio v con $low[v] \geq tin[u]$, salvo la radice che lo e se ha almeno due figli. Costo $O(n+m)$.

Grafi pesati

Se $dist[v] > dist[u] + peso(u,v)$, aggiorna $dist[v]$ e padre[v]. Questa e la operazione comune a Dijkstra e Bellman-Ford.

Funziona solo con pesi non negativi. Estrae il nodo non definitivo con distanza minima, lo rende definitivo e rilassa gli archi. Con heap: $O((n+m)\log n)$. Il motivo del fallimento con pesi negativi e che una distanza gia scelta puo essere migliorata dopo da un arco negativo.

Rilassa tutti gli archi n-1 volte; un ulteriore giro che migliora individua un ciclo negativo raggiungibile. Costo $O(nm)$, accetta pesi negativi.

$dp[k][i][j]$ e il cammino minimo da i a j usando come intermedi solo i nodi 1,...,k. Ricorrenza: $\min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])$. Costo $O(n^3)$, spazio $O(n^2)$. La diagonale negativa segnala un ciclo negativo.

MST, greedy e approssimazione

Ordina gli archi per peso e aggiungi un arco se non crea ciclo. Union-Find gestisce componenti con MakeSet, Find e Union. Costo $O(m \log m)$. La prova usa la proprieta del taglio: l'arco piu leggero che attraversa un taglio e sicuro.

Mantiene un insieme connesso e aggiunge ogni volta l'arco piu leggero che esce dall'insieme. Con heap ha costo $O((n+m)\log n)$. Non confondere MST con cammini minimi: minimizzare la somma totale non minimizza ogni cammino.

Una strategia greedy e corretta solo se si dimostra la scelta sicura e la sottostruttura ottima. Un controesempio basta a confutare una strategia, ma non basta a dimostrare una strategia corretta.

Per un algoritmo di approssimazione definire soluzione ottima OPT e soluzione prodotta A. Dimostrare $A \leq \rho \cdot OPT$ per minimizzazione, oppure $A \geq OPT/\rho$ per massimizzazione. Vertex Cover tramite un matching massimale ha fattore 2: gli archi del matching sono disgiunti, quindi il matching e un lower bound su OPT, mentre i due estremi degli archi coprono tutti gli archi.

Divide et impera

Schema: risolvere direttamente i casi piccoli; dividere; risolvere ricorsivamente; combinare. Ricerca binaria: $O(\log n)$. Merge sort: $O(n \log n)$. Massimo sottosegmento: la soluzione e tutta a sinistra, tutta a destra o attraversa il centro; combinazione lineare, costo $O(n \log n)$.

Per il k-esimo elemento, Quickselect partiziona attorno a un pivot e ricorre solo nella parte contenente k. Costo medio $O(n)$, pessimo $O(n^2)$; mediane deterministiche garantiscono $O(n)$.

La coppia di punti piu vicini divide per coordinata x. Nella striscia centrale si controllano solo un numero costante di punti per ciascuna fascia ordinata per y. Complessita $O(n \log n)$.

Per ogni ricorrenza esplicitare $T(n)$, caso base e costo della combinazione. Non applicare il teorema principale senza verificare le ipotesi.

Programmazione dinamica

La domanda decisiva e: quali sottoproblemi si ripetono e quale informazione minima descrive una loro soluzione? Scrivere sempre stato, transizione, basi, ordine di calcolo e ricostruzione.

$dp[0]=dp[1]=1$; $dp[i]=dp[i-1]+dp[i-2]$. Tempo $O(n)$, spazio $O(1)$.

$dp[i][j]$ e il massimo costo fino a (i,j) . $dp[i][j]=a[i][j]+\max(\text{sopra, sinistra})$. Tempo e spazio $O(n^2)$. Per esistenza con celle zero usare OR invece di max e scartare le celle con valore uno.

$dp[i][c]$ e il massimo valore usando i primi i oggetti con capacita c . $dp[i][c]=\max(dp[i-1][c], \text{valore}[i]+dp[i-1][c-\text{peso}[i]])$ se il peso entra. Tempo $O(nC)$, spazio $O(nC)$, ricostruzione andando a ritroso.

Se le ultime lettere coincidono, $1+dp[i-1][j-1]$; altrimenti $\max$ di $dp[i-1][j]$ e $dp[i][j-1]$. Tempo $O(pq)$. La soluzione si ricostruisce dalla cella finale seguendo le scelte.

$dp[i][j]$ e la lunghezza della sottosequenza palindroma in $s[i..j]$. Se $s[i]=s[j]$, $2+dp[i+1][j-1]$, altrimenti $\max(dp[i+1][j], dp[i][j-1])$. Riempire per diagonali. Tempo $O(n^2)$.

Il numero di alberi binari distinti con n nodi soddisfa $C[0]=1$ e $C[n]=\text{somma da } k=0 \text{ a } n-1 \text{ di } C[k]C[n-1-k]$. Tempo $O(n^2)$.

Backtracking

Il backtracking esplora un albero di configurazioni. A ogni passo scegli, controlla il vincolo, aggiungi, ricorri e annulla. La potatura deve essere giustificata: non eliminare un ramo se potrebbe contenere una soluzione.

Per stampare stringhe binarie lunghe n il numero di foglie e 2^n e il costo $O(n 2^n)$, perche ogni stringa stampata ha n simboli. Per permutazioni il costo e $O(n n!)$. Per n-regine controllare colonna, diagonale principale e secondaria.

Per problemi decisionali si puo terminare al primo successo. Per ottimizzazione si mantiene il miglior valore e si puo potare con un upper bound dimostrato.

Python: scheletri indispensabili

```
from collections import deque
import heapq
```

```
def bfs(g, s):
    d = [-1] * len(g); parent = [None] * len(g)
    q = deque([s]); d[s] = 0
    while q:
        u = q.popleft()
        for v in g[u]:
            if d[v] == -1:
                d[v] = d[u] + 1; parent[v] = u; q.append(v)
    return d, parent

def dijkstra(g, s):
    d = [float('inf')] * len(g); d[s] = 0
    heap = [(0, s)]
    while heap:
        du, u = heapq.heappop(heap)
        if du != d[u]: continue
        for v, w in g[u]:
            if d[v] > du + w:
                d[v] = du + w; heapq.heappush(heap, (d[v], v))
    return d
```

Esercizi senza soluzione

Dato un grafo non orientato, verificare in $O(n+m)$ se e un albero e spiegare perche la condizione $m=n-1$ piu connessione e sufficiente.

Dato un DAG, produrre un ordinamento topologico e restituire “impossibile” se il grafo contiene un ciclo.

Dato un grafo non pesato e due sorgenti s.t. trovare tutti i nodi equidistanti da s e t in $O(n+m)$.

Applicare Dijkstra a un grafo con almeno cinque nodi e mostrare ogni rilassamento. Poi costruire un esempio in cui un arco negativo rende errato il risultato.

Calcolare un MST con Kruskal e dimostrare, usando la proprietà del taglio, che ogni scelta è sicura.

Progettare una procedura $O(n)$ che restituisca il numero di stringhe ternarie di lunghezza n che non contengono 000, 010 e 020.

Data una matrice n per n , trovare il massimo costo da alto-sinistra a basso-destra con mosse destra o sotto. Produrre anche il cammino.

Risolvere LCS e ricostruire una sottosequenza per due stringhe date.

Generare tutte le stringhe su a, b, c con $\#a > \#b > \#c$ in $O(nS(n))$.

Risolvere n -regine e spiegare quali configurazioni vengono potate.

Scrivere e analizzare una ricorrenza per la coppia di punti più vicini.

Dimostrare il fattore 2 dell'algoritmo di Vertex Cover basato su un matching.

Soluzioni sintetiche per il controllo

Contare gli archi: se $m \neq n-1$ restituire falso; eseguire DFS/BFS da un nodo; restituire vero solo se tutti i nodi sono visitati. Costo $O(n+m)$. Un grafo connesso con $n-1$ archi e aciclico: un ciclo permetterebbe di rimuovere un arco restando connessi, ottenendo meno di $n-1$ archi.

Kahn: calcolare i gradi entranti, accodare gli zeri, rimuovere archi e inserire i nuovi zeri. Se l'output ha meno di n nodi c'è un ciclo. Costo $O(n+m)$.

Eseguire BFS da s e da t , ottenendo ds e dt ; scorrere i nodi e selezionare quelli con $ds[v]=dt[v]$. Due BFS e una scansione: $O(n+m)$.

Usare rilassamento e heap. Il controesempio minimo ha $s \rightarrow a$ di costo 1, $s \rightarrow b$ di costo 2 e $b \rightarrow a$ di costo -3: Dijkstra fissa a a 1, ma il cammino s, b, a costa -1.

Ordinare gli archi e usare Union-Find. Ogni arco aggiunto attraversa un taglio tra due componenti; essendo il più leggero che collega quelle componenti è sicuro per la proprietà del taglio. Alla fine ci sono $n-1$ archi e un albero.

Definire uno stato come il suffisso rilevante della stringa, cioè le ultime due lettere, oppure usare una DP sui prefissi con memoria del massimo suffisso che può completare un pattern vietato. Esistono un numero costante di stati, quindi ogni carattere costa $O(1)$ e il totale è $O(n)$. In alternativa costruire una macchina degli stati con suffissi "", 0, 00, 01, 02 e aggiornare i conteggi.

$dp[i][j]=a[i][j]+\max(dp[i-1][j], dp[i][j-1])$; salvare parent quando si sceglie il predecessore. Riavvolgere da $(n-1, n-1)$. Tempo $O(n^2)$, spazio $O(n^2)$.

Usare la tabella dp delle lunghezze; se le ultime lettere coincidono muovere in diagonale aggiungendo il carattere, altrimenti muovere verso la cella superiore o sinistra con valore maggiore. Tempo $O(pq)$.

Passare posizione, conteggi a, b, c ; aggiungere una lettera solo se, anche con la scelta, la condizione parziale può ancora diventare vera. Alla profondità n stampare se $a > b > c$. Ogni soluzione è generata una volta; con controllo $O(1)$ per nodo e stampa $O(n)$, costo $O(nS(n))$ se si considera l'output.

Inserire una regina per riga. Usare insiemi di colonne, $r-c$ e $r+c$; se una scelta collide, non ricorrere. Quando $r=n$, salvare la configurazione. Il test dei vincoli è $O(1)$ e la correttezza segue dal fatto che ogni disposizione possibile è costruita riga per riga.

Dividere i punti ordinati per x , risolvere le due metà, prendere delta minimo e controllare la striscia centrale ordinata per y . La combinazione è lineare, quindi $T(n)=2T(n/2)+O(n)=O(n \log n)$.

Scegliere un arco del matching, aggiungere entrambi gli estremi al cover e eliminare gli archi incidenti. Gli archi del matching sono disgiunti, dunque ogni cover deve contenere almeno un estremo per ciascuno: $|M| \leq OPT$. La soluzione prodotta ha $2|M|$ nodi, quindi è al più $2OPT$.

Checklist finale da 30 e lode

Per ogni algoritmo saper dire: problema risolto; ipotesi; invarianti; prova; tempo; spazio; caso limite; codice. Ripetere a voce la prova di BFS, Dijkstra, Kruskal e della ricorrenza DP. Saper riconoscere quando un algoritmo non si applica: Dijkstra con pesi negativi, topological sort su grafi ciclici, greedy senza proprietà di scelta sicura, DP con

stato incompleto.

Durante l'esame leggere tutto il testo prima di iniziare, indicare la tecnica scelta, dichiarare la rappresentazione e scrivere subito la complessità. Se il codice non è richiesto, lo pseudocodice chiaro e una prova completa valgono più di dettagli Python non verificati. Se il codice è richiesto, testare $n=0$, $n=1$, grafo sconnesso, archi paralleli se ammessi e assenza di soluzione.

Il materiale delle slide va usato per esercizi e notazione del docente. Questa guida non sostituisce la lettura delle tracce d'esame: serve a trasformare le slide in una mappa operativa e a concentrare il tempo sugli esercizi.

Fine della guida

Punta alla comprensione: algoritmo, prova, complessità, codice.